

Provider-Auflösung im RAG-Projekt – Workflow von .env bis ChatOpenAI

Zusammenfassung des vollständigen Wegs, wie der aktive Provider (OpenRouter oder OpenAI) vom .env-Eintrag bis zur fertigen LLM-Instanz bestimmt wird.

Gegenstück zu "modellauflösung.txt" – dort geht es um das Modell, hier um den Provider. Beide Wege laufen durch dieselbe Funktion `resolve_target()` und überschneiden sich an mehreren Stellen.

Reihenfolge der Auflösung, von der schwächsten zur stärksten Quelle:

Präfix in der Modellkennung > CHAT_PROVIDER aus .env > Klassen-Default

Übersicht

.env → Settings → `resolve_target()` → ResolvedTarget →
ChatOpenAI
 (config.py) (providers.py)
(providers.py)

Der Provider bestimmt dabei drei Dinge:

base_url -> wohin der HTTP-Request geht
key_field -> welcher SecretStr gelesen wird
env_var -> welcher Fehlertext bei fehlendem Key erscheint

Wo der Provider gesetzt werden kann

Ebene	Ort	Wirkung
-----	-----	
Grundeinstellung	config.py, Feld-Default	wenn nichts anderes gesetzt
Projektkonfig	.env, CHAT_PROVIDER	bei jedem Start
Einzellauf (Code)	ChatBot(model="openai:...")	Präfix im Argument
Einzellauf (CLI)	--model openai:gpt-4o-mini	Präfix im Argument

Station 1 – Die .env wird gelesen

Auslöser ist der erste Aufruf von `get_settings()` in `config.py`. Davor erledigt `load_environment()` das Einlesen:

```
def load_environment(dotenv_path: Path | None = None) -> bool:
    target = dotenv_path or PROJECT_ROOT / ".env"
    return load_dotenv(dotenv_path=target, override=False)
```

`override=False` heißt: Eine bereits in der Prozessumgebung gesetzte Variable gewinnt gegen die .env. In Docker oder CI ist das gewollt.

Gesucht wird die Datei über `PROJECT_ROOT`:

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]
```

Relevant für den Provider ist ein Feld in Settings:

```
chat_provider: ChatProvider = ChatProvider.OPENROUTER
```

FILTER 1 greift hier: Fehlt CHAT_PROVIDER in der .env oder ist der Wert leer, bleibt der Feld-Default stehen. Anders als bei CHAT_MODEL braucht es dafür keinen Validator – pydantic-settings ignoriert fehlende Umgebungsvariablen und verwendet den deklarierten Default.

```
.env enthält CHAT_PROVIDER nicht -> ChatProvider.OPENROUTER
.env enthält CHAT_PROVIDER=      -> ChatProvider.OPENROUTER
.env enthält CHAT_PROVIDER=openai -> ChatProvider.OPENAI
```

FILTER 2 greift bei unbekannten Werten: Ein Enum-Feld akzeptiert nur die Mitglieder von ChatProvider. Steht dort etwas anderes, wirft pydantic beim Erzeugen der Settings einen ValidationError.

```
CHAT_PROVIDER=azure -> ValidationError: Input should be 'openai' or 'openrouter'
```

Der Träger dieses Wertes ist das Enum selbst:

```
class ChatProvider(str, Enum):
    OPENAI = "openai"
    OPENROUTER = "openrouter"
```

Ergebnis dieser Station:

```
chat_provider = ChatProvider.OPENROUTER    (bei dir, da nicht gesetzt)
               = ChatProvider.OPENAI       (bei CHAT_PROVIDER=openai)
```

Station 2 – Der Ausgangswert vor der Priorisierung

In resolve_target() beginnt die Provider-Entscheidung mit dem aktiven Provider aus den Settings:

```
settings = settings or get_settings()
raw = (model or settings.chat_model or "").strip()
```

Noch ist der Provider offen – er hängt davon ab, ob raw ein Präfix trägt. Fest steht bis hierher nur der "aktive Provider":

```
provider = settings.chat_provider
```

Der Fall ist im Code noch nicht zugewiesen; die Zuweisung erfolgt je nach Zweig in Station 3 bzw. 4.

Station 3 – Weg A: Präfix in der Modellkennung

Liegt eine Kennung mit Doppelpunkt vor, entscheidet deren Präfix über den Provider – und zwar gegen `CHAT_PROVIDER`:

```
prefix = raw.partition(":")[0].lower() if ":" in raw else None
if prefix and prefix in {p.value for p in ChatProvider}:
    target = model_registry.resolve(raw)
    provider = target.provider          # <- Präfix gewinnt
else:
    provider = settings.chat_provider  # <- kein Präfix: .env entscheidet
...
```

Drei Fälle bei gesetztem Präfix:

Eingabe	Präfix gültig?	Provider
openai:gpt-4o-mini	ja	ChatProvider.OPENAI
openrouter:z-ai/glm-5.3	ja	ChatProvider.OPENROUTER
azure:gpt-4	nein	UnknownModelError

Bei ungültigem Präfix greift die Zeile nicht, der `else`-Zweig übernimmt und behandelt die ganze Kennung als Modellname ohne Präfix – was dann in der Regel an anderer Stelle scheitert.

MERKE: Wer "openai:gpt-5" schreibt, will OpenAI – auch wenn `CHAT_PROVIDER=openrouter` in der `.env` steht. Das Präfix schlägt die Konfiguration.

Station 4 – Weg B: `CHAT_PROVIDER` aus den Settings

Ohne Präfix entscheidet der Wert aus Station 1:

```
provider = settings.chat_provider
try:
    target = model_registry.get(f"{provider.value}:{raw}")
except UnknownModelError:
    target = model_registry.resolve(raw)    # Fallback: Gesamtkatalog
    provider = target.provider             # <- hier kippt der Provider
```

Der `try/except` ist der interessante Teil: Findet sich das Modell ohne Präfix nicht beim aktiven Provider, sucht der Resolver im Gesamtkatalog und übernimmt den Provider, bei dem das Modell tatsächlich liegt.

```
CHAT_PROVIDER=openrouter, CHAT_MODEL=z-ai/glm-5.3
-> erst openrouter:z-ai/glm-5.3 -> gefunden -> OpenRouter
```

```
CHAT_PROVIDER=openai,      CHAT_MODEL=z-ai/glm-5.3
-> erst openai:z-ai/glm-5.3 -> nicht gefunden
-> Fallback: z-ai/glm-5.3 liegt bei OpenRouter -> Provider kippt
```

Ist raw ganz leer, läuft es ohne Modellsuche direkt in den Default-Zweig:

```
else:
    provider = settings.chat_provider
    target = model_registry.default_for(provider)
```

Der Provider bleibt hier immer der aus der .env.

Station 5 – Konsistenzprüfung

Vor allem anderen prüft `resolve_target()`, ob Modell und Provider zusammenpassen. Dieser Check greift, wenn ein Präfix den Provider umgesetzt hat, das Modell aber nicht dort liegt:

```
if target.provider is not provider:
    raise ProviderError(
        f"Modell {target.qualified_id!r} gehoert zu "
        f"{target.provider.value!r}, aktiv ist aber {provider.value!r}. "
        f"Setze CHAT_PROVIDER={target.provider.value} oder waehle ein "
        f"Modell dieses Providers."
    )
```

In der Praxis kann diese Stelle nur erreicht werden, wenn `provider` und `target.provider` getrennt voneinander gesetzt wurden. Beide Zweige aus Station 3 und 4 setzen sie konsistent – der Check ist die Absicherung für spätere Erweiterungen.

Station 6 – Die Key-Prüfung entscheidet über den Fehlertext

Jetzt zählt sich aus, dass der Provider ein Objekt und kein String ist. Er liefert über `Properties` seinen eigenen Key-Feldnamen:

```
class ChatProvider(str, Enum):
    @property
    def key_field(self) -> str:
        return {ChatProvider.OPENAI: "openai_api_key",
                ChatProvider.OPENROUTER: "openrouter_api_key"}[self]

    @property
    def env_var(self) -> str:
        return {ChatProvider.OPENAI: "OPENAI_API_KEY",
                ChatProvider.OPENROUTER: "OPENROUTER_API_KEY"}[self]
```

Damit prüft eine generische Zeile beide Provider:

```
def _assert_key_present(provider: ChatProvider, settings: Settings) -> None:
    key_field = provider.key_field
```

```

    if getattr(settings, key_field) is None:
        raise MissingCredentialsError(
            f"{provider.env_var} is not set. "
            f"Lege den Key in der .env an: {provider.env_var}=..."
        )

```

Was daraus folgt:

Aktiver Provider	geprüftes Feld	Fehlertext bei None
openrouter	openrouter_api_key	OPENROUTER_API_KEY is not set
openai	openai_api_key	OPENAI_API_KEY is not set

WICHTIG: Die Key-Prüfung folgt IMMER dem Provider, nicht der .env-Zeile. Ein `--model openai:gpt-4o-mini` bei `CHAT_PROVIDER=openrouter` verlangt den `OPENAI_API_KEY` – obwohl die .env auf OpenRouter zeigt.

```

-----
## Station 7 – base_url und der fertige Client
-----

```

Der Provider liefert auch die Ziel-URL, ebenfalls über eine Property:

```

@property
def base_url(self) -> str:
    return {ChatProvider.OPENAI: "https://api.openai.com/v1",
            ChatProvider.OPENROUTER: "https://openrouter.ai/api/v1"}[self]

```

Damit steht das `ResolvedTarget`:

```

@dataclass(frozen=True, slots=True)
class ResolvedTarget:
    provider: ChatProvider
    model: ModelInfo
    base_url: str
    temperature: float | None
    max_tokens: int

```

`build_llm()` übersetzt das in Client-Parameter – und hier ist der Kernpunkt: ES GIBT KEINE `ChatOpenRouter`-KLASSE. Beide Provider werden über `ChatOpenAI` angesprochen, weil OpenRouter eine OpenAI-kompatible API anbietet. Der Provider unterscheidet sich nur in drei Werten:

```

kwargs = {
    "model": target.model.id,
    "base_url": target.base_url,          # <- der Provider-Unterschied
    "api_key": settings.api_key_value,    # <- welcher Key, entscheidet Station
    "max_tokens": target.max_tokens,
    "timeout": settings.request_timeout,
    "streaming": streaming,
}
if target.temperature is not None:

```

```

        kwargs["temperature"] = target.temperature

    if target.provider is ChatProvider.OPENROUTER:
        kwargs["default_headers"] = {
            "HTTP-Referer": "https://localhost/rag-project",
            "X-Title": "RAG Project Demo",
        }

    return ChatOpenAI(**kwargs)

```

Die Verzweigung nach Provider ist damit eine einzige if-Zeile für optionale Kopfzeilen. Alles andere ist datengetrieben.

 ## Die Kette in einem Durchlauf

```

1  load_environment()      .env geladen
2  Settings(...)          chat_provider = OPENROUTER          [Default,
wenn fehlt]
                             [unbekannter Wert -> ValidationError]
3  resolve_target()       raw = model or settings.chat_model or ""
4  Präfix vorhanden?
    ja -> provider = Präfix aus der Kennung    (schlägt .env)
    nein-> provider = settings.chat_provider   (+ Fallback bei
Modellfund)
5  raw leer?              default_for(provider)
6  Konsistenz             target.provider is provider?
7  Key-Prüfung            getattr(settings, provider.key_field)
8  ResolvedTarget         provider, model, base_url, temperature, max_tokens
9  build_llm()            ChatOpenAI(base_url=target.base_url,
                             api_key=..., model=target.model.id)

```

 ## Priorität in einer Zeile

Präfix im Argument > CHAT_PROVIDER > Feld-Default (OPENROUTER)

Der Lieblingsfall, der Verwirrung stiftet

CHAT_PROVIDER=openrouter in der .env, Aufruf mit --model openai:gpt-4o-mini:

```

Provider      -> OPENAI          (Präfix schlägt .env)
base_url      -> https://api.openai.com/v1
Key geprüft   -> OPENAI_API_KEY
Fehler        -> "OPENAI_API_KEY is not set"

```

Obwohl die .env auf OpenRouter zeigt. Das ist die einzige Stelle im Projekt, an der die .env den Provider nicht bestimmt.

Fehlerfälle an den Stationen

Fehler	Typ	Station
CHAT_PROVIDER unbekannt	ValidationError	Station 1
Präfix unbekannt (z. B. azure:)	UnknownModelError	Station 3
Modell nicht beim aktiven Provider	(Fallback, kein Fehler)	Station 4
Modell passt nicht zu Provider	ProviderError	Station 5
Key des aktiven Providers fehlt	MissingCredentialsError	Station 6
(Settings-Ebene, Key-Prüfung)	RuntimeError	Station 1/6

Prüfbefehle (ohne Netzwerk)

```
# Aktive Konfiguration (Keys nur als Fingerabdruck)
uv run rag-project --show-config

# Provider-Auflösung inkl. Präfix-Vorrang
uv run python -c "
from rag_project.providers import resolve_target
print('Praefix openai ->',
resolve_target('openai:gpt-4o-mini').provider.value)
print('Praefix openrouter ->',
resolve_target('openrouter:z-ai/glm-5.3').provider.value)
print('ohne Praefix ->', resolve_target().provider.value)
"

# Fehlerfall: Key des Ziel-Providers fehlt?
uv run python -c "
from rag_project.providers import resolve_target
from rag_project.config import get_settings
s = get_settings()
print('openai_api_key gesetzt :', s.openai_api_key is not None)
print('openrouter_api_key gesetzt:', s.openrouter_api_key is not None)
"
```